

MCU System with CNN Accelerator Final Project Report

1. System Architecture

本次作業整體架構由 Instruction DRAM、Data DRAM、MCU Controller、Register File、AXI Interface 以及 CNN Accelerator 所組成。Instruction DRAM 儲存程式指令，Data DRAM 儲存影像、Kernel、Weight 與一般資料。MCU 負責擷取指令、解碼、控制資料流向以及執行算術與邏輯運算，而 CNN Accelerator 則負責執行深度學習推論相關運算。透過 AXI4 Interface 作為系統匯流排，可使 MCU 與兩個 DRAM 模組進行高效率的資料交換。本設計模擬現代 SoC 中處理器與硬體加速器協同運作之情境，使整體架構更接近實際晶片設計流程。

2. MCU Control Flow and FSM Design

我使用 FSM 去進行電路實現。系統於復位後先進入 S_IDLE 狀態，完成初始化工作後開始進行 Instruction Fetch。控制器透過 S_IF_AR 與 S_IF_R 狀態向 Instruction DRAM 發出讀取請求並接收指令資料。接收到指令後進入 S_DECODE 狀態進行解碼，依據 Opcode 決定後續執行流程。對於一般算術指令，系統進入 S_EXEC 完成加法、減法與乘法等運算；對於 Load 與 Store 指令，則分別進入資料讀取與資料寫入相關狀態；對於 CNN 指令，系統進入專屬之 CNN 載入與運算流程。有限狀態機的設計使系統能夠清楚管理各階段控制訊號，並避免不同模組之間產生衝突。透過狀態分離方式，每個狀態皆負責單一功能，提升了設計可讀性與可維護性。

3. Instruction Execution Mechanism

主要處理 R-Type、I-Type、J-Type 與 E-Type 四種指令格式。R-Type 指令包含加法、減法與乘法等運算，其結果直接寫回目的暫存器。I-Type 指令包含 Load、Store 與 Branch Equal 等操作，其中 Load 與 Store 透過 AXI 介面與 Data DRAM 進行資料交換，而 Branch Equal 則根據比較結果更新 Program Counter。J-Type 指令則提供無條件跳躍功能。E-Type 指令為本專題特有之 CNN 指令，可驅動 CNN Accelerator 執行影像分類相關運算。透過這些指令，系統能夠同時具備一般微控制器與 AI 加速器控制能力。

4. AXI Interface Design

本設計遵循 AXI4 進行記憶體存取。Instruction DRAM 僅支援讀取功能，因此只需實作 Read Address 與 Read Data 通道。Data DRAM 則同時支援讀取與寫入，因此需額外實作 Write Address、Write Data 及 Write Response 通道。所有資料傳輸皆透過 valid 與 ready 訊號完成 handshake。當

Master 與 Slave 同時準備完成時，資料才會正式傳輸。

5. CNN Accelerator Integration

當 decoder 辨識到 CNN 指令後，控制器會依據指令內容選擇對應的 Image、Kernel 與 Weight。系統首先透過 AXI Burst Read 從 Data DRAM 將資料載入至內部 Buffer，包括兩張影像的 Channel 資料、CNN core 以及 fully connect weight。完成 load 後，控制器逐筆將資料送入 CNN 模組。CNN Accelerator 會輸出四個分類結果，控制器收集所有輸出後進行比較，找出最大值所對應之索引，最後將分類結果寫回指定暫存器。透過此流程，系統成功整合先前實作之 CNN Accelerator，形成完整的 MCU-CNN 協同運算平台。

6. Burst Transfer Optimization

由於 CNN 指令在執行過程中需要從 Data DRAM 讀取大量資料，包括兩張影像的雙通道資料、卷積核心以及全連接層權重，因此記憶體存取延遲將成為整體系統效能的重要瓶頸。

為了解決問題，我利用 AXI4 協定所提供的 Burst Transfer 機制，將連續位址空間中的資料一次性搬移至本地緩衝區(Buffer)。根據題目所提供的記憶體配置方式，每張影像包含兩個通道(Channel)，每個通道各有 36 筆 8-bit 資料，因此單張影像共需讀取 72 Bytes。每組卷積核心則包含兩個通道，各有 9 個權重，因此總共需要 18 Bytes。全連接層權重則包含 32 個 8-bit 資料，共需讀取 32 Bytes。由於這些資料在 DRAM 中皆以連續位址方式儲存，非常適合 Burst Read 進行存取。

在系統執行 CNN 指令時，控制器會根據目前欲讀取的資料種類設定不同的 Burst Length。對於影像資料，系統一次讀取完整的 72 Bytes，因此設定 AXI Burst Length 為 71；對於 Kernel 資料，則一次讀取 18 Bytes，因此設定 Burst Length 為 17；對於 Weight 資料，則一次讀取 32 Bytes，因此設定 Burst Length 為 31。由於 AXI 協定中的 Burst Length 為實際傳輸次數減一，因此上述設定剛好對應各資料區塊的大小。透過此方式，系統僅需發送一次 Read Address Request，便能連續取得整個資料區塊，大幅減少重複 handshake 所造成的時間浪費。

除了基本的 Burst Transfer 機制之外，本設計進一步針對影像資料進行最佳化。由於題目中的影像資料是依序存放於記憶體中，因此當指令指定的兩張影像滿足 $\text{imgB} = \text{imgA} + 1$ 時，代表兩張影像在 DRAM 中實際上是連續儲存的資料區塊。在此情況下，若仍然採用兩次獨立 Burst Read，則必須進

行兩次 Address Handshake；然而本設計會偵測此條件並將兩次存取合併為一次較大的 Burst Transaction。原本需要分別讀取的兩個 72 Bytes 資料區塊，將合併成一次 144 Bytes 的連續讀取，因此只需設定 Burst Length 為 143 即可完成兩張影像的載入。透過此方法，系統能進一步減少 AXI Address Transaction 的數量，降低記憶體控制開銷。

Burst Transfer 透過一次傳輸大量連續資料，不僅降低了 AXI 通訊開銷，也大幅減少了 CNN Accelerator 等待資料的時間。此外，針對連續影像所設計的合併讀取機制，更進一步提升了資料搬移效率，使整個 MCU 與 CNN Accelerator 的協同運作更加流暢，進而提高系統整體吞吐量與執行效能。

7. Out-of-Order in CNN Latency Hiding and Simplified Execution

CNN Accelerator 的運算延遲遠大於一般指令，若處理器在發出 CNN 指令後完全等待結果產生，將造成大量時脈週期浪費。為提升系統效能，本設計導入簡化版 Out-of-Order Execution 概念。當 CNN 指令送出後，處理器不會立即停止，而是允許後續與 CNN 結果無資料相依性的指令繼續執行。系統利用 cnn_pending、cnn_busy 以及 cnn_rd 等訊號建立簡易 Scoreboard 機制，用以追蹤 CNN 指令之執行狀態與目的暫存器。當後續指令嘗試讀取尚未完成之 CNN 結果時，控制器才會進行 Stall。此機制能有效重疊 CPU 執行時間與 CNN 計算時間，達到 Latency Hiding 效果。

8. Conclusion

此次成功實作一套整合 MCU 與 CNN Accelerator 的完整系統。設計中包含 Instruction Fetch、Instruction Decode、Register File、AXI Interface 與 CNN Accelerator 等關鍵模組，並透過有限狀態機完成整體控制流程。此外，本設計進一步利用 Burst Transfer 與簡化版 Out-of-Order Execution 概念提升系統效能，使 CNN 運算延遲得以與 CPU 執行流程重疊。透過本專題的實作，不僅加深了我對處理器架構與 AXI 協定的理解，也實際體驗了現代 SoC 與 AI Accelerator 整合設計流程